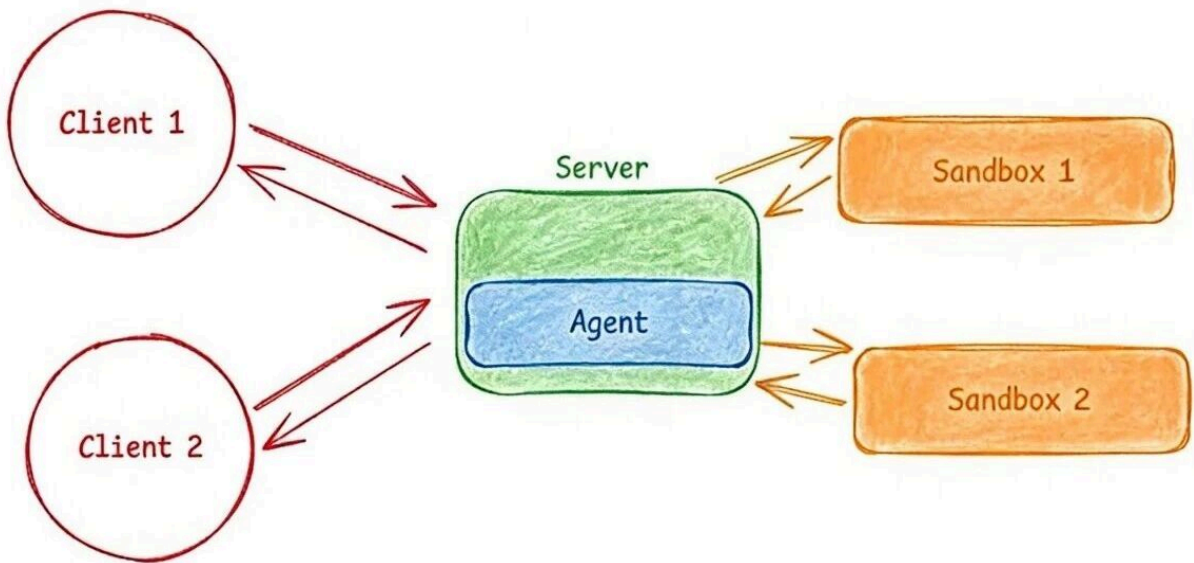


Agent 如何连接 Sandbox ? 两种主流架构模式详解

mp.weixin.qq.com/s/30i7yY1zqAm8e5uHy-lotQ

零度AI



分享作者：Harrison Chase (LangChain 创始人，deepagents 相关项目推动者)

在最近的一篇技术分享中，Harrison 系统性总结了 Agent 与 Sandbox 集成的两种核心架构模式。随着越来越多的 AI Agent 需要运行代码、安装依赖、读写文件，“给 Agent 一个安全的工作空间”已经成为基础能力，而 Sandbox 正是这个安全边界。

本文将对这两种模式进行结构化拆解，并结合实际工程经验，帮助你在构建 Agent 系统时做出正确选择。

一、为什么 Agent 一定需要 Sandbox ?

越来越多的 Agent 不再只是“对话工具”，而是具备：

- 执行 Python / Shell 代码
- 安装第三方依赖
- 访问和修改文件
- 调用外部 API

这本质上意味着：Agent 需要一台“电脑”。

但问题是——如果这台“电脑”没有隔离，Agent 可能：

- 读取你的 API Key
- 访问主机文件系统
- 被 prompt injection 利用窃取敏感信息
- 执行恶意命令

因此我们需要 Sandbox（沙箱环境）来隔离 Agent 运行环境与主机系统。

本文讨论的是“完整计算环境型 Sandbox”，例如 Docker 容器或虚拟机，不包括语言级或进程级沙箱。

问题不在于“要不要 Sandbox”，而是：

Agent 应该运行在 Sandbox 里面，还是把 Sandbox 当成工具远程调用？

这就是下面两种模式。

二、模式一：Agent 运行在 Sandbox 内部 (Agent IN Sandbox)

架构结构

Agent 直接运行在 Sandbox 里面。

你通过网络与它通信（HTTP / WebSocket）。

[你的应用] <——网络通信——> [Sandbox 内运行的 Agent]

实际部署方式

1. 构建 Docker 或 VM 镜像
2. 在镜像内预装 Agent 框架
3. 在 Sandbox 内启动 Agent 服务
4. 对外暴露 API 接口
5. 通过网络发送消息与 Agent 交互

优势

1 与本地开发体验高度一致

如果你在本地终端运行 deepagents :

```
deepagents run
```

那么在 Sandbox 中运行的命令几乎一致。

生产环境 $\approx$ 本地环境

这对调试和环境一致性非常友好。

2 强耦合场景更适合

如果 Agent :

- 需要长期维护复杂环境状态
- 依赖特定系统库
- 频繁读写本地文件
- 修改环境配置

这种模式会更自然。

缺点与风险

! 1. 通信层需要额外基础设施

你必须：

- 构建 WebSocket / HTTP 服务
- 管理 session
- 处理错误
- 维护连接

部分厂商（如 E2B）提供 SDK 封装，但不是所有平台都有。

! 2. API Key 必须放在 Sandbox 内

这意味着：

- 如果 Sandbox 被攻破
- 或发生 prompt injection
- 可能导致密钥泄露

目前 E2B、Runloop 等在开发 Secret Vault 机制缓解此问题。

! 3. 更新成本高

每次修改 Agent 逻辑：

- 需要重建镜像
- 重新部署

迭代速度变慢。

! 4. IP 泄露风险更高

如果 Agent 在 Sandbox 内运行，

攻击者更容易导出：

- 完整代码
- Prompt
- 内部逻辑

! 5. 权限控制粒度较粗

Witan Labs 的 Nuno Campos 提出一个关键问题：

| Agent in Sandbox 模式下，LLM 生成的代码权限 == Agent 权限。

例如：

- Agent 有 bash 权限
- Agent 有 web fetch 工具

那么 LLM 生成代码可以无限制调用 web fetch。

但如果是 Sandbox 作为工具模式：

- 安全边界围绕“执行工具”
- 你可以给工具不同权限
- LLM 生成代码权限更可控

这是非常重要的架构差异。

三、模式二：Sandbox 作为工具 (Sandbox as Tool)

架构结构

Agent 运行在本地或服务器上。

当需要执行代码时，调用远程 Sandbox API。

[本地 Agent] ——调用 API——> [远程 Sandbox 执行代码]

实际工作流程

当 Agent 执行任务时：

1. 在本地进行推理和规划
2. 生成 Python 代码
3. 调用 Sandbox Provider API
4. 远程执行代码
5. 返回结果
6. Agent 继续本地推理

优势

1 快速迭代

修改 Agent 逻辑：

- 不需要重建镜像
- 不需要重新部署 Sandbox

开发效率大幅提高。

2 API Key 保持在主环境

只有代码执行在 Sandbox 内。

推理和密钥都在主环境。

安全性更高。

3 关注点分离 (Separation of Concerns)

Agent 状态：

- 对话历史
- 记忆
- 推理链

运行在主环境。

Sandbox 只负责：

- 执行代码
- 隔离运行

好处：

- Sandbox 崩溃不会丢失 Agent 状态
- 可以轻松切换不同 Sandbox Provider

4 并行能力更强

E2B 的 Tomas Beran 提出：

- 可以同时启动多个远程 Sandbox
- 支持并行任务

适合批量执行或大规模任务。

5 成本控制更优

只在执行代码时付费。

不需要为整个 Agent 生命周期支付资源。

Ben Guo 提出一个前瞻观点：

未来可能：

- Agent 推理在 GPU 机器
- Sandbox 在 CPU 隔离环境

两者环境需求会分化。

模式二更适合这种趋势。

缺点

！ 网络延迟

每次执行：

- 都跨网络边界
- 小任务频繁调用时延迟明显

部分 Sandbox 提供“状态会话”机制：

- 变量持久
- 文件持久
- 依赖持久

可以减少调用次数。

四、如何选择？

选择模式一（Agent IN Sandbox）如果：

- Agent 与环境高度耦合
- 需要复杂长期环境状态
- 希望生产环境与本地完全一致
- Provider 已帮你处理通信层

选择模式二（Sandbox as Tool）如果：

- 需要快速迭代 Agent
- 不希望 API Key 放入 Sandbox
- 希望更清晰的权限控制
- 需要并行执行能力
- 希望更好扩展性

五、deepagents 的支持

deepagents 是一个开源 Agent 框架，
同时支持两种模式。

模式一：构建镜像后在 Sandbox 内运行。

模式二：将 Sandbox 作为 backend 工具调用。

这使得团队可以：

- 先用模式二快速开发
- 在特定场景迁移到模式一

- 或混合使用

六、架构趋势总结

从系统设计角度来看：

维度	Agent IN Sandbox	Sandbox as Tool
安全性	密钥在 Sandbox	密钥在主环境
迭代速度	较慢	快
架构复杂度	通信复杂	调用简单
权限控制	粗粒度	细粒度
并行能力	一般	强
未来 GPU 适配	较弱	强

七、最终结论

Agent 必须运行在隔离环境中，这是共识。

但真正的架构选择在于：

你是让 Agent 住进 Sandbox ，

| 还是把 Sandbox 变成 Agent 的工具？

如果你追求：

- 快速开发
- 清晰边界
- 安全隔离
- 可扩展架构

大多数情况下，模式二（Sandbox as Tool）会成为主流选择。

如果你的系统高度依赖环境状态或需要强耦合运行环境，模式一仍然有其价值。

随着 Agent 系统复杂度提升，架构设计能力将成为核心竞争力。

而这两种模式，是所有 Agent 系统设计者必须理解的基础分叉点。